

数字电路与逻辑设计

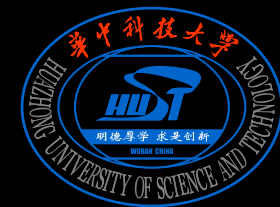
第 2 章 逻辑代数与硬件描述语言基础

张江山

zhangjs@hust.edu.cn

信息工程系

2 逻辑代数与硬件描述语言基础



- 2.1 逻辑代数的基本定理和恒等式
- 2.2 逻辑函数表达式的形式
- 2.3 逻辑函数的代数化简法
- 2.4 逻辑函数的卡诺图化简法
- 2.5 硬件描述语言 Verilog HDL 基础

1. 掌握逻辑代数常用基本定律、恒等式和规则
2. 掌握逻辑函数的基本表达式及相互转换，代数化简方法
3. 掌握逻辑函数最小项和最大项定义及性质，卡诺图化简法
4. 掌握硬件描述语言 Verilog HDL
5. 掌握正、负逻辑运算
6. 掌握逻辑符号的等效变换

2.1 逻辑代数的基本定理和规则

● 逻辑代数：又称布尔代数

- ◆是分析和设计现代数字逻辑电路的数学工具
- ◆逻辑代数的定律和规则，用于对表达式进行处理，以完成对逻辑电路的化简、变换、分析和设计

● 逻辑关系

- ◆是事件产生的条件和结果之间的因果关系
- ◆在数字电路中，将事件的条件作为输入信号，结果作为输出信号
- ◆条件和结果中两种对立状态分别用逻辑 **1** 和 **0** 表示

2.1.1 逻辑代数的基本定律和恒等式

基本定律	与	或
1. 0—1 律	$A \cdot 0 = 0, A \cdot 1 = A$	$A + 0 = A, A + 1 = 1$
2. 重叠律	$A \cdot A = A$	$A + A = A$
3. 互补律	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
4. 结合律	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
5. 交换律	$A \cdot B = B \cdot A$	$A + B = B + A$
6. 分配律	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + B \cdot C = (A + B) \cdot (A + C)$
7. 反演律	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$
8. 吸收律	$A \cdot (A + B) = A$ $A \cdot (\bar{A} + B) = A \cdot B$	$A + A \cdot B = A$ $A + \bar{A} \cdot B = A + B$
常用恒等式	$A \cdot B + \bar{A} \cdot C + B \cdot C = A \cdot B + \bar{A} \cdot C$ $A \cdot B + \bar{A} \cdot C + B \cdot C \cdot D = A \cdot B + \bar{A} \cdot C$	

证明等式成立: $A \cdot B + \bar{A} \cdot C + B \cdot C = A \cdot B + \bar{A} \cdot C$

证: 左式 = $A \cdot B + \bar{A} \cdot C + (A + \bar{A}) \cdot B \cdot C$

$$= A \cdot B + \bar{A} \cdot C + A \cdot B \cdot C + \bar{A} \cdot B \cdot C$$

$$= A \cdot B \cdot (1 + C) + \bar{A} \cdot C \cdot (1 + B) = A \cdot B + \bar{A} \cdot C$$

2.1.1 逻辑代数的基本定律和恒等式

等式证明

(1) 代数法：代数变换

(2) 穷举法

将等式两边分别用 F_1 , F_2 表示,
穷举输入变量所有取值的组合,
分别代入 F_1 和 F_2 算出相应的结果,
若都相等, 则 $F_1 = F_2$, 否则, $F_1 \neq F_2$

用真值表法证明摩根定理

$$\text{证: } \overline{A + B} = \overline{A} \cdot \overline{B}$$

$$\text{令: } F_1 = \overline{A + B}, F_2 = \overline{A} \cdot \overline{B}$$

A	B	F_1	F_2
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

得证

2.1.1 逻辑代数的基本定律和恒等式

例：证明 $A + \bar{A} \cdot B = A + B$

$$\begin{aligned} A + \bar{A} \cdot B &= A \cdot (1 + B) + \bar{A} \cdot B \\ &= A + A \cdot B + \bar{A} \cdot B \\ &= A + (A + \bar{A}) \cdot B \\ &= A + B \end{aligned}$$

例：试化简下列逻辑函数 $L = (A + B) \cdot (\bar{A} + B)$

$$\begin{aligned} \text{由分配率 } L &= A \cdot \bar{A} + A \cdot B + \bar{A} \cdot B + B \cdot B \\ &= 0 + A \cdot B + \bar{A} \cdot B + B \\ &= B \cdot (A + \bar{A} + 1) \\ &= B \end{aligned}$$

2.1.2 逻辑代数的基本规则

1. 代入规则

- **规则**：任何含有某变量的等式，若等式中所有出现该变量的位置均代之以一个逻辑函数式，则此等式依然成立
- **作用**：扩大基本公式的应用范围

例如，根据反演律 $\overline{A \cdot B} = \overline{A} + \overline{B}$ BC 代替 B

得： $\overline{A \cdot B \cdot C} = \overline{A} + \overline{B \cdot C}$ 利用摩根定律
$$= \overline{A} + \overline{B} + \overline{C}$$

由此，摩根定律能推广至 n 个变量：

$$\overline{A_1 \cdot A_2 \cdot \dots \cdot A_n} = \overline{A_1} + \overline{A_2} + \dots + \overline{A_n}$$

$$\overline{A_1 + A_2 + \dots + A_n} = \overline{A_1} \cdot \overline{A_2} \cdot \dots \cdot \overline{A_n}$$

2.1.2 逻辑代数的基本规则

2. 反演规则

- **规则**：对于任意一个逻辑函数式 F ，做如下处理，得到的新函数式称为原函数式 F 的反函数式
 - ◆ 若把式中的运算符 $\cdot$ 换成 $+$ ， $+$ 换成 $\cdot$
 - ◆ 常量 0 换成 1 ， 1 换成 0
 - ◆ 原变量换成反变量，反变量换成原变量
 - ◆ 保持原函数的运算次序不变
- **作用**：求原函数式 F 的反函数式 $\bar{F}$

例 2.1.1 试求 $L = \bar{A} \cdot \bar{B} + C \cdot D + 0$ 的反函数

解：根据反演规则，得 $\bar{L} = (A + B) \cdot (\bar{C} + \bar{D}) \cdot 1$
$$= (A + B) \cdot (\bar{C} + \bar{D})$$

2.1.2 逻辑代数的基本规则

注意事项

- (1) 保持原函数的运算次序不变，必要时适当地加入括号
- (2) 不属于单个变量上的非号有两种处理方法：
 - 非号保留，而非号下面的函数式按反演规则变换
 - 将非号去掉，而非号下的函数式保留不变

例如 $F(A, B, C) = A \cdot \bar{B} + \overline{(A + C)} \cdot B + \bar{A} \cdot \bar{B} \cdot \bar{C}$

其反函数为 $\bar{F} = (\bar{A} + B) \cdot \overline{\bar{A} \cdot \bar{C} + B} \cdot (A + B + C)$

或 $\bar{F} = (\bar{A} + B) \cdot (A + C) \cdot B \cdot (A + B + C)$

将 $(A + C) \cdot B$ 看成一个变量 P

3. 对偶规则

- **对偶式**：任意逻辑函数式 F ，做如下处理，得到的新函数式称为原函数式 F 的对偶式 F' ，也称对偶函数
 - ◆ 若把式中的运算符 $\cdot$ 换成 $+$ ， $+$ 换成 $\cdot$
 - ◆ 常量 0 换成 1 ， 1 换成 0
 - ◆ 保持原函数的运算次序不变
- **规则**：若 $F_1 = F_2$ ，则 $F_1' = F_2'$
- **作用**：使定理公式的证明步骤减半

例：逻辑函数 $L = (A + \bar{B}) \cdot (A + C)$ 的对偶式为：

$$L' = A \cdot \bar{B} + A \cdot C$$

2.1.2 逻辑代数的基本规则



注意事项：保持原函数的运算次序不变，必要时适当地加入括号

1. 例如：
$$F = \overline{A \cdot B + \overline{A} \cdot C + 1 \cdot B}$$

其对偶式
$$F' = (\overline{A + B}) \cdot (\overline{\overline{A} + C}) \cdot (\overline{0 + B})$$

2. 已知 $A + \overline{A} \cdot B = A + B$ ，求证 $A \cdot (\overline{A} + B) = A \cdot B$

解：令 $F_1 = A + \overline{A} \cdot B$ ， $F_2 = A + B$

则 F_1 和 F_2 的对偶式分别为： $F_1' = A \cdot (\overline{A} + B)$ ， $F_2' = A \cdot B$

由于 $F_1 = F_2$ ，故 $F_1' = F_2'$

2.1.2 逻辑代数的基本规则

逻辑函数有不同形式

与 - 或式 $A \cdot C + \bar{C} \cdot D$

与非 - 与非式 $\overline{\overline{A \cdot C} \cdot \overline{\bar{C} \cdot D}}$ (摩根定理可证与上式等价)

或 - 与式

$$\begin{aligned} & (A + \bar{C}) \cdot (C + D) \\ &= A \cdot C + A \cdot D + \bar{C} \cdot C + \bar{C} \cdot D \\ &= A \cdot C + A \cdot D \cdot (C + \bar{C}) + 0 + \bar{C} \cdot D \\ &= A \cdot C + A \cdot C \cdot D + A \cdot \bar{C} \cdot D + \bar{C} \cdot D \\ &= A \cdot C + \bar{C} \cdot D \end{aligned}$$

或非 - 或非式 $\overline{\overline{A + \bar{C}} + \overline{\bar{C} + D}}$

与 - 或 - 非式 $\overline{\overline{A \cdot C} + \overline{\bar{C} \cdot D}}$

任何表达式都可变换为上述五种形式的表达式

最简与或表达式：与项数最少，且各与项中变量数最少的与 - 或表达式

2.2 逻辑函数形式 2.2.2 最小项表达式

1. 最小项的定义和性质

最小项： n 个变量 $X_1, X_2, \dots, X_n$ 的最小项，是 n 个因子的与项，各变量必须都以其原或反变量的形式在与项中出现且仅出现一次

n 个变量的最小项应有 2^n 个

三个变量 A, B, C 有八个 (2^3) 最小项：

$\bar{A}\bar{B}\bar{C}, \bar{A}\bar{B}C, \bar{A}B\bar{C}, \bar{A}BC, A\bar{B}\bar{C}, A\bar{B}C, AB\bar{C}, ABC$

二个变量 A, B 有四个 (2^2) 最小项： $\bar{A}\bar{B}, \bar{A}B, A\bar{B}, AB$

一个变量 A 有二个 (2^1) 最小项： $\bar{A}, A$

对于三个变量 A, B, C 来说， $\bar{A}B, A\bar{B}C\bar{A}, A(B+C)$ 是不是最小项？

都不是

2.2.2 最小项表达式

2. 最小项的性质

- (1) 使某最小项 m_i 为 1 的变量值仅有一组，且该组变量值拼接成的数为 i
- (2) 不同的最小项，使其等于 1 的那一组变量的取值也各不相同
- (3) $m_i \cdot m_j = 0$ ($i \neq j$)
- (4) 全部最小项之或结果为 1

三变量的最小项

A B C	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	$F = \sum_{i=0}^{2^n-1} m_i$
	$\bar{A} \bar{B} \bar{C}$	$\bar{A} \bar{B} C$	$\bar{A} B \bar{C}$	$\bar{A} B C$	$A \bar{B} \bar{C}$	$A \bar{B} C$	$A B \bar{C}$	$A B C$	
<u>0 0 0</u>	1	0	0	0	0	0	0	0	1
<u>0 0 1</u>	0	1	0	0	0	0	0	0	1
0 1 0	0	0	1	0	0	0	0	0	1
0 1 1	0	0	0	1	0	0	0	0	1
1 0 0	0	0	0	0	1	0	0	0	1
1 0 1	0	0	0	0	0	1	0	0	1
1 1 0	0	0	0	0	0	0	1	0	1
1 1 1	0	0	0	0	0	0	0	1	1

2.2.2 最小项表达式

3. 最小项表达式

- 由若干最小项构成的与 - 或表达式，也称为标准与 - 或式
 - ◆ 为“与或”逻辑表达式
 - ◆ 在“与或”式中的每个与项都是最小项
- 任何逻辑函数都可变换成唯一的最小项表达式
- 转换的方法有：(1) 代数转换法，(2) 真值表转换法

例 1：求 $L(A, B, C) = AB + \bar{A}C$ 的最小项表达式

解： $L(A, B, C) = AB(C + \bar{C}) + \bar{A}(B + \bar{B})C$

缺啥补啥

$$= ABC + AB\bar{C} + \bar{A}BC + \bar{A}\bar{B}C$$

$$\quad \quad \quad 1 \ 1 \ 1 \quad 1 \ 1 \ 0 \quad 0 \ 1 \ 1 \quad 0 \ 0 \ 1$$

$$= m_7 + m_6 + m_3 + m_1$$

$$= \sum m(7, 6, 3, 1)$$

2.2.2 最小项表达式

(1) 代数转换法

第一步：将逻辑函数转换成一般与或表示式

第二步：反复使用 $X = X(Y + \bar{Y})$ ，将所有非最小项的与项变为最小项

例：

$$L(A, B, C) = \overline{(AB + \bar{A}\bar{B} + C)} \cdot \bar{A}\bar{B}$$

第一步

$$= \overline{(AB + \bar{A}\bar{B} + C)} + AB = \bar{A}\bar{B} \cdot \bar{A}\bar{B} \cdot C + AB$$

摩根定理

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

$$= (\bar{A} + \bar{B})(A + B)C + AB = (\bar{A}B + A\bar{B})C + AB$$

$$= \bar{A}BC + A\bar{B}C + AB$$

第二步

$$= \bar{A}BC + A\bar{B}C + AB(C + \bar{C})$$

$$= \bar{A}BC + A\bar{B}C + ABC + AB\bar{C}$$

$$= m_3 + m_5 + m_7 + m_6 = \sum m(3, 5, 6, 7)$$

2.2.2 最小项表达式

(2) 真值表转换法——基本思想

最小项表达式是若干最小项构成的，若某组变量值使表达式 $F=1$ ，则 F 中一定有该组变量值所对应的结果为 1 的最小项

例如，已知两变量逻辑函数 F 真值表如下

A	B	F	m_i
0	0	0	$m_0 = 1$
0	1	1	$m_1 = 1$
1	0	1	$m_2 = 1$
1	1	0	$m_3 = 1$

2 变量共有 4 个最小项：

$$m_0 = \bar{A}\bar{B}, m_1 = \bar{A}B$$

$$m_2 = A\bar{B}, m_3 = AB$$

$$F = m_0 ? + m_1 ? + m_2 ? + m_3 ?$$

A, B 取 0, 0 时: $m_0 = 1$

而此时 $F=0$ ，故 F 中不能包含 m_0

A, B 取 0, 1 时: $m_1 = 1$

而此时 $F=1$ ，故 F 中包含 m_1

...

$$\therefore F = m_1 + m_2 = \bar{A}B + A\bar{B}$$

2.2.2 最小项表达式

在举重比赛中有三个裁判员，只有当两个或两个以上裁判员认为杠铃已经举起时，才算是成功，试写出逻辑表达式

输入： A, B, C 0 表示认为失败， 1 表示认为成功

输出： F 0 表示失败， 1 表示成功

列真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$F = \bar{A} B C + A \bar{B} C + A B \bar{C} + A B C$$

$$\bar{A} B C = m_3$$

$$A \bar{B} C = m_5$$

$$A B \bar{C} = m_6$$

$$A B C = m_7$$

2.2.2 最小项表达式

(2) 真值表转换法——方法

第一步：将逻辑函数转换成一般与或表示式

第二步：列真值表，写出标准与或表示式

例 $F(A, B, C) = A\bar{B} + B\bar{C}$

已是与或式

第二步：列真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

A, B 取 1, 0 时: $F = 1$

B, C 取 1, 0 时: $F = 1$

其余填 0

故 F 中包含四个最小项: m_2, m_4, m_5, m_6

$$F = m_2 + m_4 + m_5 + m_6 = \sum m(2, 4, 5, 6)$$

2.2.3 最大项表达式

1. 最大项的定义

- n 个变量 $X_1, X_2, \dots, X_n$ 的最大项, 是 n 个因子的或项, 各变量都以原变量或非变量的形式在或项中出现且仅出现一次
- n 个变量的最大项应有 2^n 个

例如, A, B, C 三个逻辑变量的最大项有 $2^3 = 8$ 个, 即:

$$\bar{A} + \bar{B} + \bar{C}, \quad \bar{A} + \bar{B} + C, \quad \bar{A} + B + \bar{C}, \quad \bar{A} + B + C,$$

$$A + \bar{B} + \bar{C}, \quad A + \bar{B} + C, \quad A + B + \bar{C}, \quad A + B + C$$

2.2.3 最大项表达式

2. 最大项的性质

- (1) 使某最大项 M_i 为 0 的变量值仅有一组，且该组变量值拼接成的数为 i
- (2) 不同的最大项，使其等于 0 的那一组变量的取值也各不相同
- (3) $M_i + M_j = 1$ ($i \neq j$)
- (4) 全部最大项之和结果为 1

3. 最小项和最大项的关系

两者之间为互补关系： $m_i = \overline{M_i}$ ， 或者 $M_i = \overline{m_i}$

比如逻辑函数 $F(A, B)$ 中， A, B 取值为

$$00 \text{ 时: } m_0 = \overline{A} \overline{B} = 1, M_0 = A + B = 0$$

$$01 \text{ 时: } m_1 = \overline{A} B = 1, M_1 = A + \overline{B} = 0$$

$$10 \text{ 时: } m_2 = A \overline{B} = 1, M_2 = \overline{A} + B = 0$$

$$11 \text{ 时: } m_3 = A B = 1, M_3 = \overline{A} + \overline{B} = 0$$

2.2.3 最大项表达式

例：逻辑电路的真值表如右，写出最小项和最大项表达式

●最小项表达式： $L = 1$ 的各个最小项之或

$$L(A, B, C) = m_3 + m_5 + m_6$$

$$= \sum m(3, 5, 6)$$

$$= \bar{A}BC + A\bar{B}C + AB\bar{C}$$

$$\bar{L}(A, B, C) = m_0 + m_1 + m_2 + m_4 + m_7$$

$$\bar{\bar{L}}(A, B, C) = \overline{m_0 + m_1 + m_2 + m_4 + m_7}$$

$$= \overline{m_0} \cdot \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_7}$$

$$= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \cdot M_7$$

●最大项表达式： $L = 0$ 的各个最大项之与

$$L(A, B, C) = M_0 \cdot M_1 \cdot M_2 \cdot M_4 \cdot M_7$$

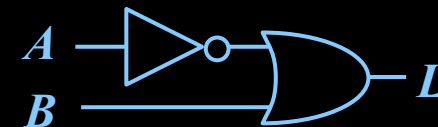
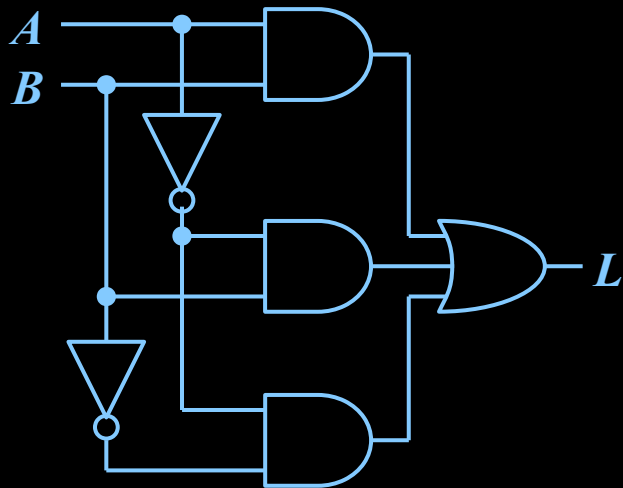
$$= \prod M(0, 1, 2, 4, 7)$$

$$= (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(\bar{A} + B + C)(\bar{A} + \bar{B} + \bar{C})$$

A	B	C	L	$\bar{L}$
0	0	0	0 M_0	1 m_0
0	0	1	0 M_1	1 m_1
0	1	0	0 M_2	1 m_2
0	1	1	1 m_3	0
1	0	0	0 M_4	1 m_4
1	0	1	1 m_5	0
1	1	0	1 m_6	0
1	1	1	0 M_7	1 m_7

2.3 代数法化简

化简的目的：降低电路成本，以较少的门实现电路



两图电路逻辑功能相同

$$L = A B + \overline{A} B + \overline{A} \overline{B}$$

$$L = \overline{A} + B$$

$$= A B + \overline{A} (B + \overline{B})$$

$$= A B + \overline{A} (B + 1)$$

$$= A B + \overline{A} B + \overline{A}$$

$$= B + \overline{A}$$

2.3 代数法化简

函数式变换的目的，除了化简，也可减少门电路的种类

通常在一片集成电路芯片中只有一种门电路

例：已知 $L = AB\bar{D} + \bar{A}\bar{B}\bar{D} + ABD + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD$

(1) 求最简的与 - 或式，并画出相应的逻辑图

(2) 画出仅用与非门实现的电路

解： $L = AB(\bar{D} + D) + \bar{A}\bar{B}\bar{D} + \bar{A}\bar{B}D(\bar{C} + C)$

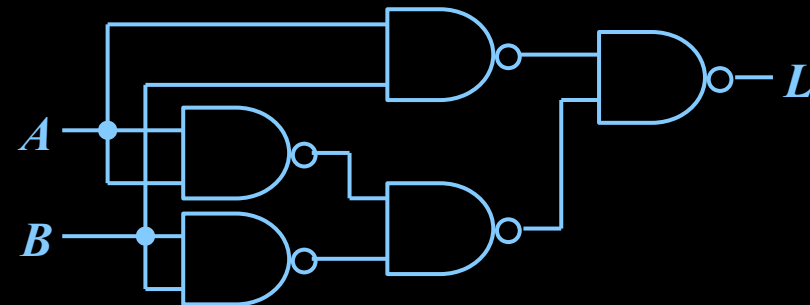
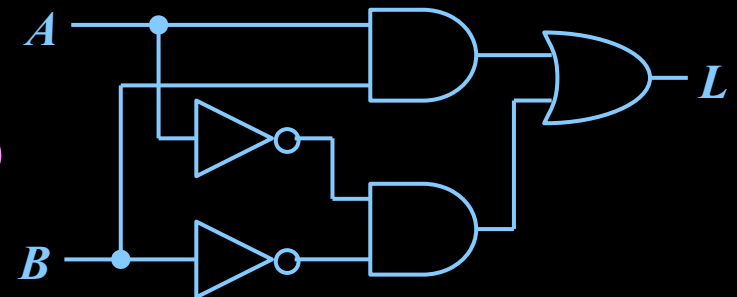
$$= AB + \bar{A}\bar{B}\bar{D} + \bar{A}\bar{B}D$$

$$= AB + \bar{A}\bar{B}(D + \bar{D})$$

$$= AB + \bar{A}\bar{B}$$

$$= \overline{\overline{AB} + \overline{\bar{A}\bar{B}}}$$

$$= \overline{\overline{AB} \cdot \overline{\bar{A}\bar{B}}}$$



2.4 卡诺图化简法 2.4.1 用卡诺图表示逻辑函数

1. 卡诺图

将 n 变量的最小项分别用 2^n 个方格表示，变量值按格雷码顺序依次作为各方格的行、列坐标，以使逻辑相邻最小项对应的方格在几何位置上也相邻，由此得到的图形称为 n 变量的卡诺图

逻辑相邻： 仅一个变量互反的最小项是逻辑相邻的

例如三变量最小项： $m_6 = A B \bar{C}$ 与 $m_7 = A B C$ 是逻辑相邻的

几何相邻： 卡诺图位置毗邻或以对称轴对褶后相重的方格是几何相邻的

两变量卡诺图

A \ B	0	1
0	m_0	m_1
1	m_2	m_3

三变量卡诺图

A \ BC	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

四变量卡诺图

AB \ CD	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

2.4.1 用卡诺图表示逻辑函数

2. 卡诺图特点

n 个变量的最小项 m_i , 有 n 个相邻项

相邻的最小项可合并

$$m_{12} + m_{14} = A B \bar{C} \bar{D} + A B C \bar{D}$$

$$= A B \bar{D} (\bar{C} + C)$$

$$= A B \bar{D}$$

消除互反因子,
保留公因子

例：找出 m_{12} 和 m_2 的相邻项

L AB \ CD				
	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

$$m_{12} = \underline{A} \underline{B} \underline{\bar{C}} \underline{\bar{D}}$$

$$m_4 = \underline{\bar{A}} \underline{B} \underline{\bar{C}} \underline{\bar{D}}$$

$$m_8 = A \underline{\bar{B}} \underline{\bar{C}} \underline{\bar{D}}$$

$$m_{13} = A B \underline{\bar{C}} \underline{D}$$

$$m_{14} = A B \underline{C} \underline{\bar{D}}$$

2.4.1 用卡诺图表示逻辑函数

3. 逻辑函数的卡诺图

- 当逻辑函数为最小项表达式时，将表达式中出现的最小项对应的方格标 1，其余方格标 0（也可不标），便得到逻辑函数的卡诺图
- 任何逻辑函数等于卡诺图中标 1 格对应最小项之或，标 0 格对应最大项之与

例如 画出 $L(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 8, 10, 11, 14, 15)$ 的卡诺图

$$L(A, B, C, D) = \prod M(5, 6, 7, 9, 12, 13)$$

L CD AB		00	01	11	10
		00	01	11	10
标 1 格	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

注：函数卡诺图中 0 可不填

标 0 格

2.4.1 用卡诺图表示逻辑函数

卡诺图是真值表的一种平面几何图形表示方法

若一组变量值使逻辑函数 F 为 1，则该组值便是其卡诺图中标 1 格的坐标

例如 $F = \sum m(1, 2, 3, 7) = \underline{\bar{A}\bar{B}C} + \underline{\bar{A}B\bar{C}} + \underline{\bar{A}BC} + \underline{ABC}$

函数 F 真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

	BC	00	01	11	10
F	A				
0		0	1	1	1
1		0	0	1	0

函数 F 的卡诺图

2.4.1 用卡诺图表示逻辑函数

逻辑函数 F 的卡诺图表示步骤：

1. 求逻辑函数 F 的与或式
2. 根据变量数确定卡诺图形式
3. 根据各与项，在卡诺图上标 1

例如 $F = \underline{A B} + \underline{C D} + \underline{\bar{A} \bar{B} C}$

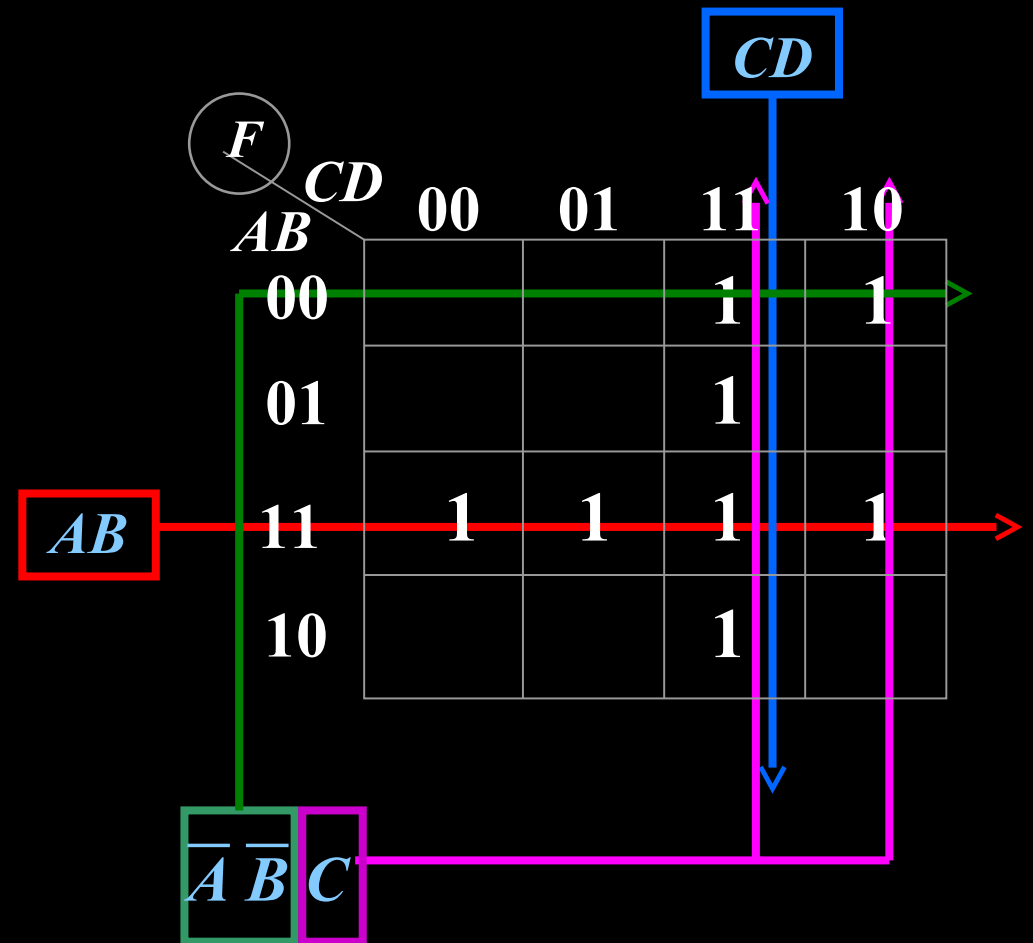
1. 已是与或式
2. 根据变量数确定卡诺图形式
3. 根据各与项，在卡诺图上标 1

(1) $A = 1, B = 1, F = 1$

(2) $C = 1, D = 1, F = 1$

(3) $A = 0, B = 0, C = 1, F = 1$

其余标 0，也可不标，即得到 F 的卡诺图



2.4.1 用卡诺图表示逻辑函数

例：画出 $L(A, B, C, D) = \prod M(0, 6, 10, 13, 15)$ 的卡诺图

解： $L = (A + B + C + D)(A + \bar{B} + \bar{C} + D)(\bar{A} + B + \bar{C} + D)(\bar{A} + \bar{B} + C + \bar{D})(\bar{A} + \bar{B} + \bar{C} + \bar{D})$

根据反演规则

$$\bar{L} = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + A\bar{B}C\bar{D} + AB\bar{C}D + ABCD = \sum m(0, 6, 10, 13, 15)$$

画出 $\bar{L}$ 卡诺图

$\bar{L}$ $AB \backslash CD$	00	01	11	10
00	1	0	0	0
01	0	0	0	1
11	0	1	1	0
10	0	0	0	1

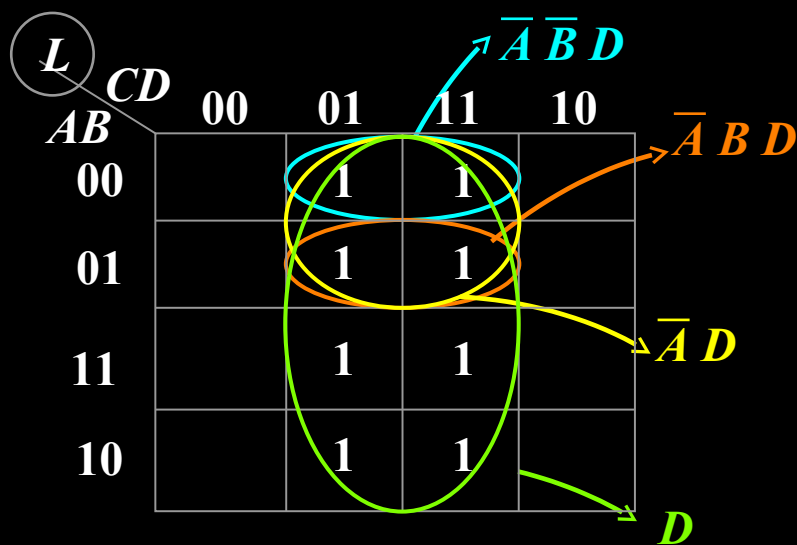
L 卡诺图

L $AB \backslash CD$	00	01	11	10
00	0	1	1	1
01	1	1	1	0
11	1	0	0	1
10	1	1	1	0

也可用在 L 中出现的最大项标 0，未出现的最大项标 1，直接画出 L 卡诺图

2.4.2 用卡诺图化简逻辑函数

1. 卡诺图化简的依据：逻辑函数中的相邻最小或最大项是可合并的



消除互反变量，保留公因子

$$L = m_1 + m_3 = \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD = \bar{A}\bar{B}D$$

$$L = m_5 + m_7 = \bar{A}B\bar{C}D + \bar{A}BCD = \bar{A}BD$$

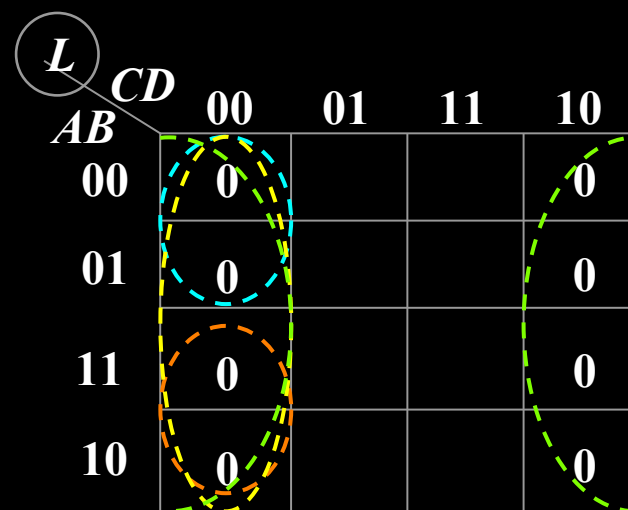
$$L = m_1 + m_3 + m_5 + m_7 = \bar{A}\bar{B}D + \bar{A}BD = \bar{A}D$$

$$L = \sum m(1, 3, 5, 7, 9, 11, 13, 15) = D$$

2 个相邻最小项合并，消去 1 个变量

2^2 个相邻最小项合并，消去 2 个变量

2^3 个相邻最小项合并，消去 3 个变量



$$\begin{aligned} L &= M_0 \cdot M_4 \\ &= (A + B + C + D) \cdot (A + \bar{B} + C + D) \\ &= (A + C + D) + B \cdot \bar{B} \quad \text{或的分配率} \\ &= A + C + D \end{aligned}$$

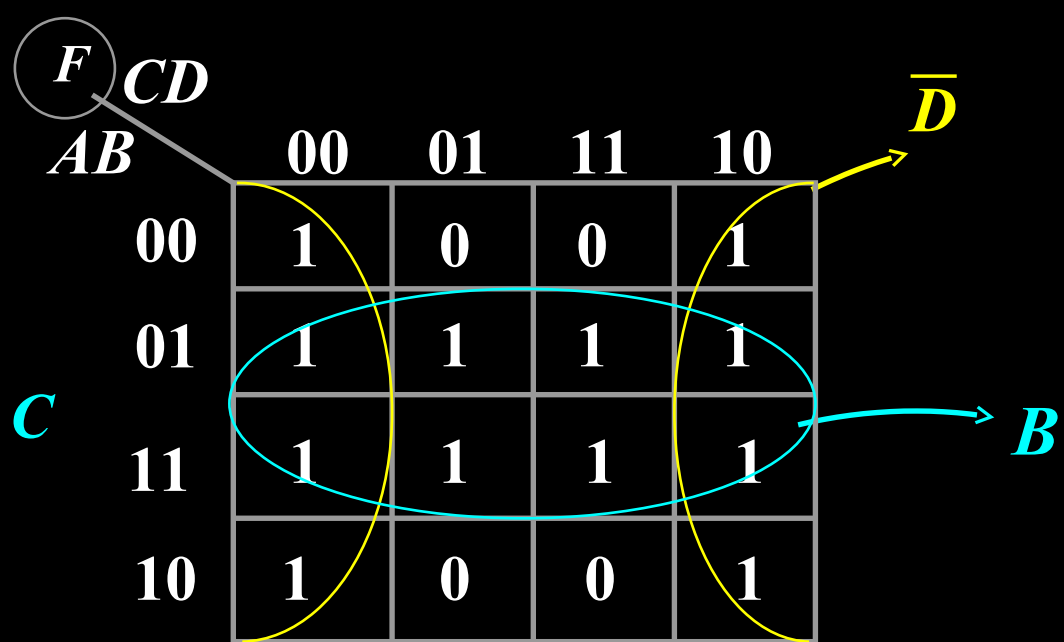
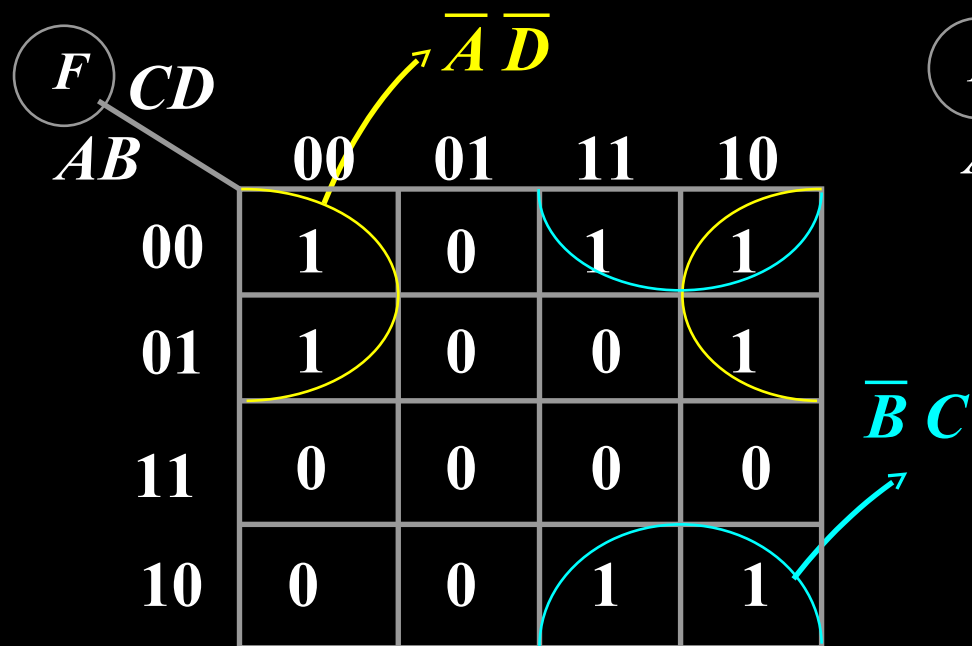
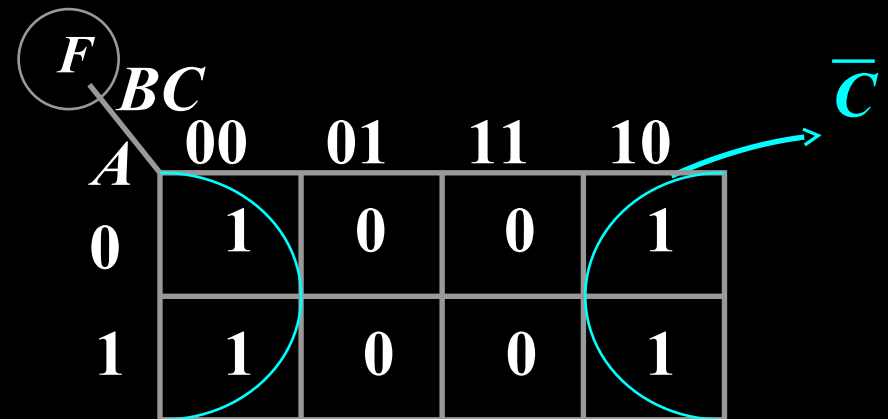
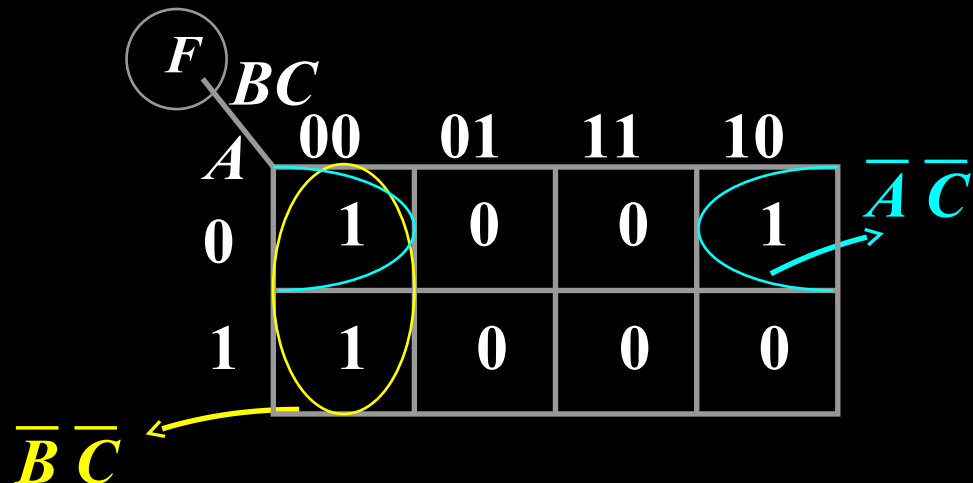
$$\begin{aligned} L &= M_{12} \cdot M_8 \\ &= (\bar{A} + \bar{B} + C + D) \cdot (\bar{A} + B + C + D) \\ &= \bar{A} + C + D \end{aligned}$$

$$\begin{aligned} L &= M_0 \cdot M_4 \cdot M_{12} \cdot M_8 \\ &= (A + C + D) \cdot (\bar{A} + C + D) \\ &= C + D \end{aligned}$$

$$L = \prod M(0, 2, 4, 6, 8, 10, 12, 14) = D$$

2.4.2 用卡诺图化简逻辑函数

- 任何逻辑函数等于其卡诺图中标 1 格所对应的最小项之或
- 若将卡诺图中标 1 格所对应的最小项合并，则达到化简的目的



2.4.2 用卡诺图化简逻辑函数

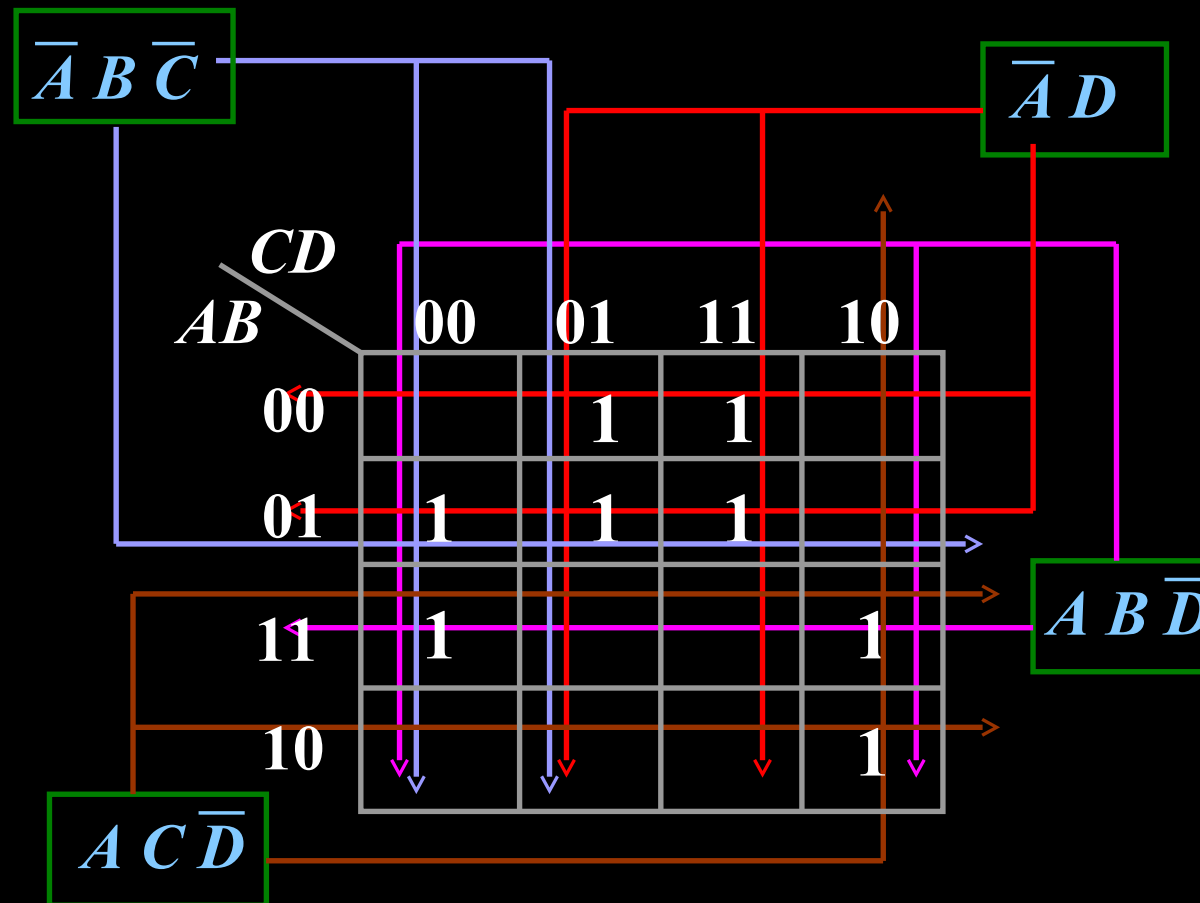
2. 卡诺图化简的步骤

- (1) 将逻辑函数转换成与 - 或表达式
- (2) 根据逻辑函数与 - 或表达式在卡诺图中标 1，得到逻辑函数的卡诺图
- (3) 画卡诺圈合并与项原则： 2^k 个相邻标 1 格圈入一个卡诺圈
 - 该卡诺圈即被合并为消除了 k 个因子的与项，标 1 格可重复被圈入
 - 新增卡诺圈直至所有标 1 格均已被圈入，圈数越少，则与项数越少
- (4) 由上所得各卡诺圈对应与项之或，即为最简与或表达式

2.4.2 用卡诺图化简逻辑函数

例：用卡诺图化简 $F = \underline{A B \bar{D}} + \underline{\bar{A} D} + \underline{\bar{A} B \bar{C}} + \underline{A C \bar{D}}$

(1) 画出逻辑函数的卡诺图



F 的卡诺图

2.4.2 用卡诺图化简逻辑函数

例：用卡诺图化简 $F = A B \bar{D} + \bar{A} D + \bar{A} B \bar{C} + A C \bar{D}$

(2) 根据最小项合并原则画卡诺圈

(3) 各卡诺圈对应的与项之或，即最简与或表达式

$AB \backslash CD$		CD			
		00	01	11	10
00			1	1	
01	1		1	1	
11	1				1
10					1

$$F = \bar{A} D + A C \bar{D} + B \bar{C} \bar{D}$$

2.4.2 用卡诺图化简逻辑函数

例：用卡诺图化简

$$L(A, B, C, D) = \sum m(0..3, 5..11, 13..15)$$

解：画出函数 L 的卡诺图

L $AB \backslash CD$	00	01	11	10
00	1	1	1	1
01	0	1	1	1
11	0	1	1	1
10	1	1	1	1

$$L = \overline{B} + C + D$$

画出函数 $\overline{L}$ 的卡诺图

$\overline{L}$ $AB \backslash CD$	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	0	0	0
10	0	0	0	0

$$\overline{L} = B \overline{C} \overline{D}$$

$$L = \overline{B \overline{C} \overline{D}} = \overline{B} + C + D$$

● 可用圈 0 法先求出反函数最简与或式，再求反

2.4.2 用卡诺图化简逻辑函数

例：已知 $G(A, B, C, D) = A\bar{C} + \bar{C}D + \bar{A}C\bar{D} + \bar{B}C\bar{D}$

$H(A, B, C, D) = (A + B + \bar{C} + \bar{D})(\bar{B} + \bar{C} + D)(\bar{A} + C + \bar{D})$

求 $F(A, B, C, D) = G \cdot H$ 的最简与或式

解： $\bar{H}(A, B, C, D) = \bar{A}\bar{B}CD + BCD\bar{D} + A\bar{C}D$ (反演律)

$$\bar{H}$$

CD \ AB	00	01	11	10
00	0	0	1	0
01	0	0	0	1
11	0	1	0	1
10	0	1	0	0

$$H$$

CD \ AB	00	01	11	10
00	1	1	0	1
01	1	1	1	0
11	1	0	1	0
10	1	0	1	1

$$G$$

CD \ AB	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	1

$$G \cdot H$$

CD \ AB	00	01	11	10
00	0	1	0	1
01	0	1	0	0
11	1	0	0	0
10	1	0	0	1

故 $F = A\bar{C}\bar{D} + \bar{A}\bar{C}D + \bar{B}C\bar{D}$

2.4.2 用卡诺图化简逻辑函数

3. 含无关项的逻辑函数化简

- 某些输入变量值可能与实际逻辑问题无关，被约束限制
 - ◆ 比如：输入 8421BCD 码时，不能输入 1010、1011、1100、1101、1110、1111
- 输入变量具有无关条件的逻辑问题，称为包含无关条件的逻辑问题
- 这些无关条件的变量值对应的最小项称为无关项或任意项，用 **d** 表示
- 卡诺图化简时，这些无关项可根据实际化简目的取任意值
 - ◆ 但要确保不能输入无关条件的变量值，否则函数值不可用

2.4.2 用卡诺图化简逻辑函数

例：设计电路，能判断一位十进制数是奇数还是偶数

奇数时，电路输出为 1，为偶数时，电路输出为 0

解：

(1) 列出真值表

(2) 画卡诺图化简

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>L</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	d
1	0	1	1	d
1	1	0	0	d
1	1	0	1	d
1	1	1	0	d
1	1	1	1	d

无关条件为： $AB + AC = 0$

<i>L</i> <i>CD</i> <i>AB</i>		00	01	11	10
		00	01	11	10
00		0	1	1	0
01		0	1	1	0
11		d	d	d	d
10		0	1	d	d

必须填写无关项
否则可能错当 0

输入变量须满足无关条件
即须确保电路不输入这些无关信号

2.4.2 用卡诺图化简逻辑函数

例：已知 $F(A, B, C, D) = \bar{A}B\bar{C} + \bar{A}\bar{C}D + C\bar{D}(A \oplus B)$ ，无关条件为： $AB + CD = 0$ ，求 F 的最简与或式、或与式。

解：

(1) 写出 F 的一般与或式

$$\begin{aligned} F &= \bar{A}B\bar{C} + \bar{A}\bar{C}D + C\bar{D}(\bar{A}B + A\bar{B}) \\ &= \bar{A}B\bar{C} + \bar{A}\bar{C}D + \bar{A}BC\bar{D} + A\bar{B}C\bar{D} \end{aligned}$$

(2) 画出 F 的卡诺图

(3) 由输入约束条件，标记无关项

(4) 画出卡诺圈

(5) 写出最简与或式

(6) 写出最简或与式

F		CD			
		00	01	11	10
AB	00		1	d	
	01	1	1	d	1
	11	d	d	d	d
	10			d	1

$$F = B + \bar{A} \cdot D + A \cdot C$$

F		CD			
		00	01	11	10
AB	00	0	1	d	0
	01	1	1	d	1
	11	d	d	d	d
	10	0	0	d	1

$$F = (\bar{A} + C) \cdot (A + B + D)$$

●HDL (Hardware Description Language)

- ◆类似于高级程序语言，文本形式描述数字电路结构和行为的语言
- ◆可描述逻辑电路图，逻辑表达式，信号及逻辑电路功能行为

●逻辑仿真（前仿真，行为仿真）

- ◆用计算机仿真软件对数字逻辑电路的结构和行为进行预测
- ◆仿真器对 **HDL** 描述进行解释，以文本或波形描述电路输出

●逻辑综合

- ◆根据 **HDL** 描述导出电路基本元件表及其连接关系（门级网表）的过程
- ◆类似高级程序语言的编译过程
- ◆根据网表数据库，可制作出集成电路或印刷电路板 **PCB**

●时序仿真（后仿真）

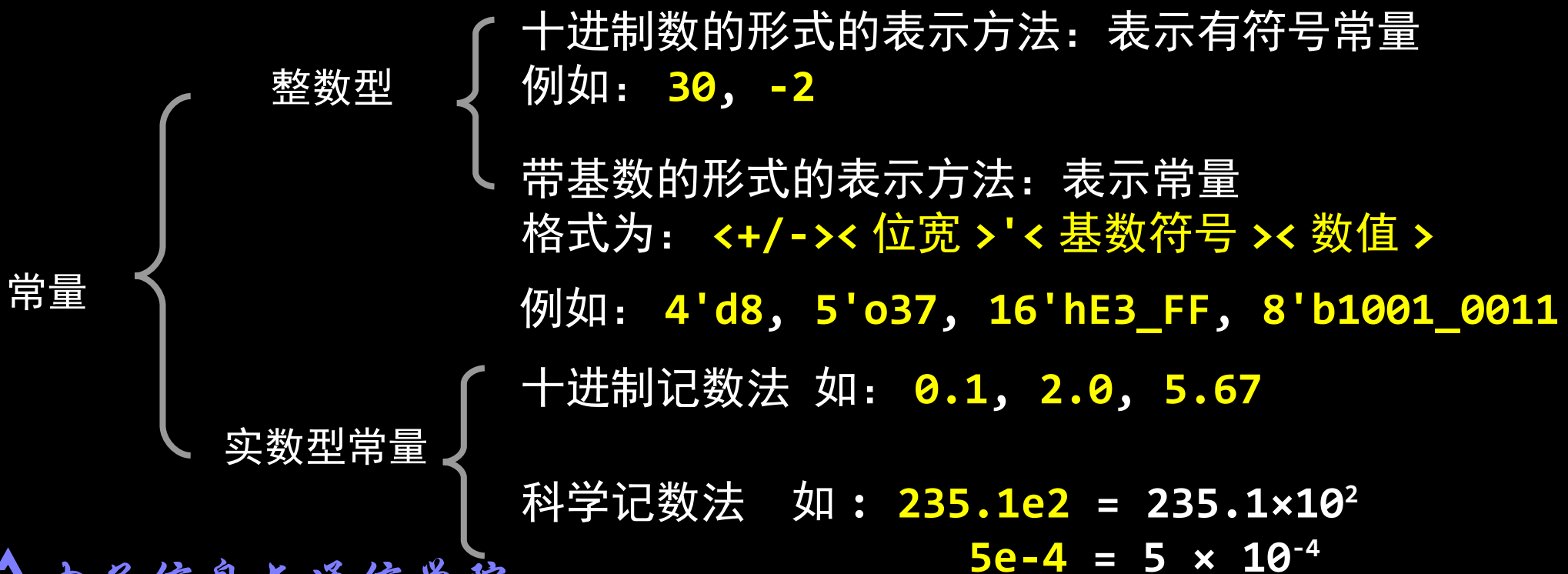
2.5.1 Verilog HDL 语言的基本语法规则

- 建模：用规定的语法规则描述数字电路的过程
- 间隔符：分隔文本
 - ◆ 如：空格符（**\b**）、TAB 键（**\t**）、页符
- 注释符：改善可读性，不影响编译
 - ◆ 多行注释符（用于写多行注释）：**/* --- */**
 - ◆ 单行注释符：以 **//** 开始到行尾结束为注释文字
- 标识符：用字符串标识对象（如模块、输入 / 输出端口、变量）
 - ◆ 以英文字母或下划线开始
 - ◆ 如 **clk** , **counter8** , **_net** , **bus_A**
- 关键词：用来定义语言结构的特殊词汇
 - ◆ 关键词都是小写，关键词不能作为标识符使用
 - ◆ 如 **module**, **endmodule**, **input**, **output**, **wire**, **reg**, **and** 等

2.5.1 Verilog HDL 语言的基本语法规则

- 逻辑值集合： Verilog 语言规定了 4 种基本的逻辑值

0	逻辑 0，逻辑假
1	逻辑 1，逻辑真
x 或 X	不确定的值（未知状态）
z 或 Z	高阻态



2.5.2 数据类型

- 参数：用标识符代表常量，又称符号常量

parameter 参数名 1 = 常量表达式 1, 参数名 2 = 常量表达式 2, ... ;

如: **parameter** BIT = 1, BYTE = 8, PI = 3.14;

- 字符串：字符串是双撇号内的字符序列

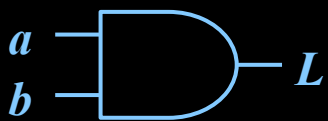
- 变量：

◆ 线网型: **wire** <[n-1: 0]> 变量名 1, 变量名 2, ... ;

◆ 寄存型: **reg** <[n-1: 0]> 变量名 1, 变量名 2, ... ;

➤ 只可在 **initial** 或 **always** 语句块内被赋值

➤ 行为描述是电路功能行为的抽象描述，并不描述电路的具体组成结构



```
wire a, b, L;           // 1-bit 线网型变量
wire[7:0] DataBus;      // 8-bit 线网型总线变量
reg clock;              // 1 位寄存型变量
reg[3:0] counter;       // 4 位寄存型变量
```

类型	说明
reg	1 位寄存型变量
integer	32 位带符号的整数寄存型变量
real	64 位带符号的实数寄存型变量
time	64 位无符号的时间寄存型变量

2.5.3 运算符及其优先级

1. 运算符

{ } { }	Concatenation, replication	拼接, 复制
unary + unary -	Unary operators	单目正负
+ - * / **	Arithmetic	算术
%	Modulus	模
> >= < <=	Relational	关系
!	Logical negation	逻辑非
&&	Logical and	逻辑与
	Logical or	逻辑或
==	Logical equality	逻辑相等
!=	Logical inequality	逻辑不等
=== 全等	Case equality	(including x and z)
!= 不全等	Case inequality	(including x and z)

A = 4'b11zx;
 B = 4'b1110;
 (A == A) 和 (A == B) 结果都是 x
 (A === A) 结果是 1, (A === B) 结果是 0

~	Bitwise negation	位非
&	Bitwise and	位与
	Bitwise inclusive or	位或
^	Bitwise exclusive or	位异或
^~ or ~^	Bitwise equivalence	位同或
&	Reduction and	单目缩位与
~&	Reduction nand	单目缩位与非
	Reduction or	单目缩位或
~	Reduction nor	单目缩位或非
^	Reduction xor	单目缩位异或
~^ or ^~	Reduction xnor	单目缩位同或
<<<	Logical left shift	逻辑左移
>>>	Logical right shift	逻辑右移
<<<<	Arithmetic left shift	算术左移
>>>>	Arithmetic right shift	算术右移
?:	Conditional	条件

位拼接运算符

将两个或多个信号的某些位拼接起来成为一个新的操作数

设 $A = 1'b1$, $B = 2'b10$, $C = 2'b00$

$\{B, C\} = 4'b10_00$

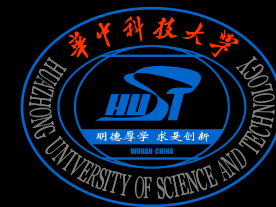
$\{A, B[1], C[0]\} = 3'b1_1_0$

$\{A, B, C, 3'b101\} = 8'b1_10_00_101$

对同一个操作数的重复拼接还可以双重大括号构成的运算符 $\{ \{ \} \}$

例如 $\{4\{A\}\} = 4'b1111$

$\{\{2\{A\}\}, \{2\{B\}\}, C\} = 8'b11_1010_00$



位运算与缩位运算

A : 4'b1010

B : 4'b1111

A: 1010

B: 1111

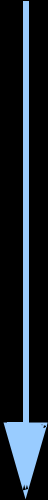
&
——
1010

A & B : A 与 B 进行位运算

& A : A 进行缩位运算, $1 \& 0 \& 1 \& 0 = 0$

2.5.3 运算符及其优先级

2. 运算符的优先级

+ - ! ~ & ~& ~ ^ ~^ ^~ (unary)	Highest precedence
**	
* / %	
+ - (binary)	
<< >> <<< >>>	
< <= > >=	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
 (binary)	
&&	
 	
?: (conditional operator)	
{ } { } { }	Lowest precedence

2.5.4 Verilog 内部的基本门级元件

门级建模：将逻辑电路图用 HDL 规定的文本语言表示出来

基本门级元件模型

多输入门	and	与门	nand	与非门
	or	或门	nor	或非门
	xor	异或门	xnor	异或非门
多输出门	buf	缓冲器	not	反相器
三态门	bufif1	高电平有效控制三态缓冲器	notif1	高电平有效控制三态反相器
	bufif0	低电平有效控制三态缓冲器	notif0	低电平有效控制三态反相器

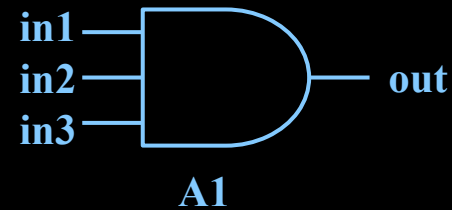
2.5.4 Verilog 内部的基本门级元件

1. 多输入门

只允许有一个输出，但可以有多多个输入

and A1(out, in1, in2, ...);

标识符



or O1(out, in1, in2, ...);

xor X1(out, in1, in2, ...);

and

in1 in2	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

or

in1 in2	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

xor

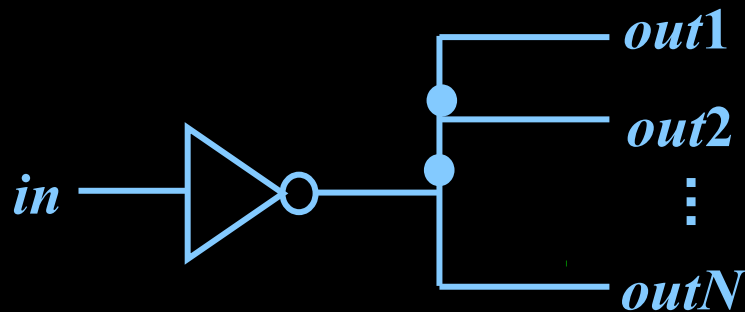
in1 in2	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

2.5.4 Verilog 内部的基本门级元件

2. 多输出门

允许有多个输出，但只有一个输入

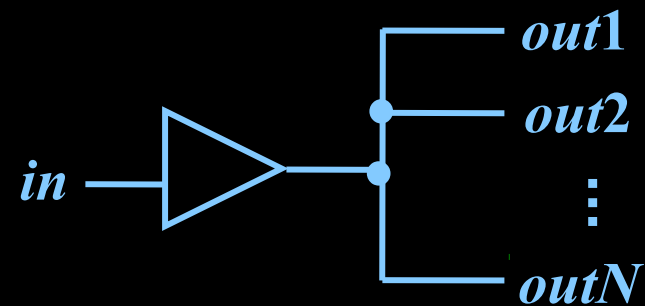
`not N1(out1, out2, ..., in);`



not

in	out
0	1
1	0
x	x
z	x

`buf B1(out1, out2, ..., in);`

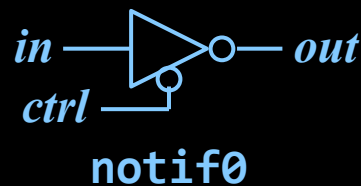
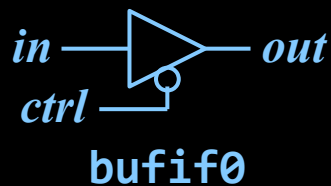
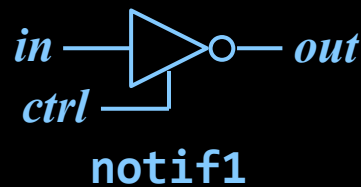
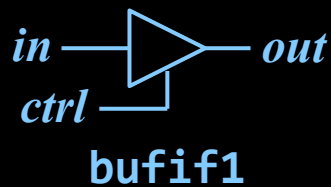


buf

in	out
0	0
1	1
x	x
z	x

3. 三态门

有输出、数据输入和输入控制端各 1 个，若控制信号无效，则输出高阻态 z



bufif1

ctrl	in	out
0	0	z
0	1	z
0	x	z
0	z	z
1	0	0
1	1	1
1	x	x
1	z	x

notif1

ctrl	in	out
0	0	z
0	1	z
0	x	z
0	z	z
1	0	1
1	1	0
1	x	x
1	z	x

● 模块是 Verilog 描述电路的基本单元

```
module ABC(port1, port2, ...); // 各模块通过端口相互连接
    inout/input/output ... ;    // 端口类型说明

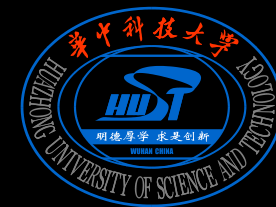
    reg/wire/ ... ;             // 变量类型说明

    parameter ... ;             // 参数定义，逗号分割多个参数

    and/or/ ... ;               // 结构描述方式，逻辑门或底层模块

    assign ... ;                 // 数据流描述方式，连续赋值语句

    always/initial               // 行为描述方式，过程块结构
    begin                         // 多条语句组成的行为描述块
        if ... else ... ;       // 条件描述语句
        case                     // 分支描述语句
            ...
        endcase                  // 无实际语义的语句无分号
    end
endmodule
```



- Verilog 2005 标准新增的参数和端口定义形式

```
module MEM #(parameter dataWIDTH = 32, addrWIDTH = 32)
    (output reg[dataWIDTH-1:0] data, input[addrWIDTH-1:0] addr);
    ...
endmodule
```

- 新增的模块使用形式

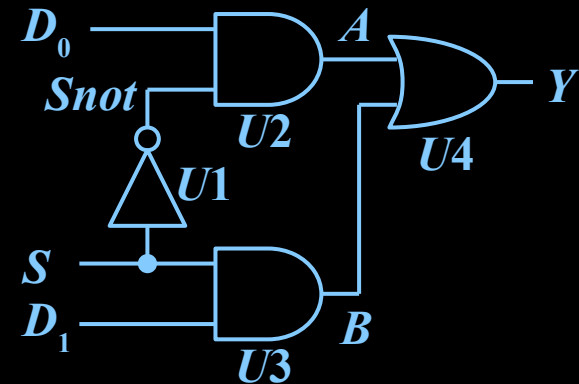
```
wire[7:0] addr_bus;
wire[31:0] data_bus;
MEM #(.addrWIDTH(8)) U1(.addr(addr_bus), .data(data_bus));
```

2.5.5 Verilog 程序的基本结构

例 用结构描述方式建立门电路 Verilog 模型

已知电路逻辑结构时，
用门级建模

模块名



```
module mux2to1(output Y, input[1:0] D, input S);
```

```
  wire Snot, A, B; // 定义内部节点信号数据类型
```

数据类型说明

```
  not U1(Snot, S);
```

```
  and U2(A, D[0], Snot);
```

```
  and U3(B, D[1], S);
```

```
  or U4(Y, A, B);
```

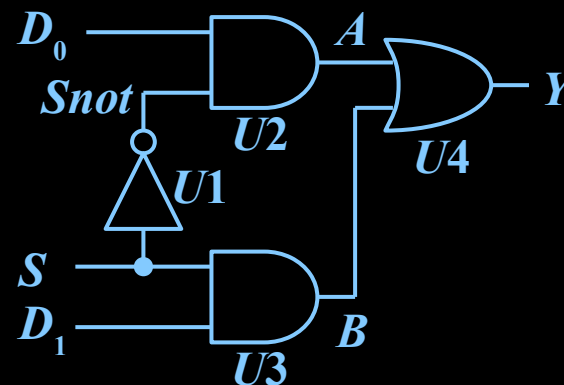
用门级建模方式
描述电路结构

```
endmodule
```

例 用数据流描述方式建立模型

$$Y = D_0 \cdot \bar{S} + D_1 \cdot S$$

已知逻辑表达式时，
用数据流建模



```
module mux2to1(output Y, input[1:0] D, input S);
```

```
    assign Y = (~S & D[0]) | (S & D[1]);
```

```
endmodule
```

- **assign** 语句，被连续赋值的变量（等号左边）必须是 **wire** 型
- 端口变量默认为 **wire** 型

例 用行为描述方式建模，实现二选一选择器电路

已知电路行为功能要求时，
用行为级建模

```
module mux2to1(output reg Y, input[1:0] D, input S);  
    always @(S, D)  
        if(S == 1) Y = D[1];  
        else Y = D[0];  
endmodule
```

- **always** 和 **initial** 语句块中，被过程赋值的变量类型必须是寄存型

编写 Testbench 测试向量

```
`include "mux2to1.v"           // 预编译包含被测试模块描述文件
`timescale 1us/100ns           // 预编译定义仿真单位时间 / 精度
module testbench;              // 定义测试模块
    reg[1:0] D;                 // 定义测试信号变量
    reg S;
    wire Y;
    mux2to1 test(Y, D, S);      // 接入被测模块, 将测试信号接入端口
    initial begin               // 初始化行为描述 (仅用于仿真, 不可综合)
        // 根据指定信号事件, 格式化输出字符串 (系统任务或函数以 $ 标识)
        $monitor($time, ": D0 D1 S Y -> %d %d %d %d", D[0], D[1], S, Y);
        $dumpfile("testbench.vcd"); // 设置信号输出文件
        $dumpvars(0, testbench);    // 指定输出的信号范围 (范围, 模块)
        $display("Running testbench"); // 格式化输出字符串, 类似 printf
        #20 $finish;                // 20 个单位时间后, 结束仿真
    end
    initial begin               // 初始化行为描述, 各描述块之间是独立、无序的
        D = 2'b00;
        S = 0;
        #10 S = 1;               // 10 个单位时间后, S 信号置 1
    end
    always #1 D[0] = ~D[0];       // 周期性行为描述, 各描述块之间是独立、无序的
    always #5 D[1] = ~D[1];      // D[0] 周期 2us, D[1] 周期 10us
endmodule
```

2.5.6 逻辑功能的仿真与测试

1. 安装 iverilog

```
sudo apt install iverilog, gtkwave
```

2. 编译

```
iverilog testbench.v -o testbench.o
```

3. 行为仿真（前仿真）

```
vvp -n testbench.o      # 非单步仿真
```

4. 波形分析

```
gtkwave testbench.vcd
```

5. 综合：用支持所选器件的综合工具输出网表数据

6. 时序仿真（后仿真）：不同器件时延参数不同，可能影响结果

7. 下载网表数据至所选器件

3.7 逻辑描述中的几个问题 3.7.1 正负逻辑问题

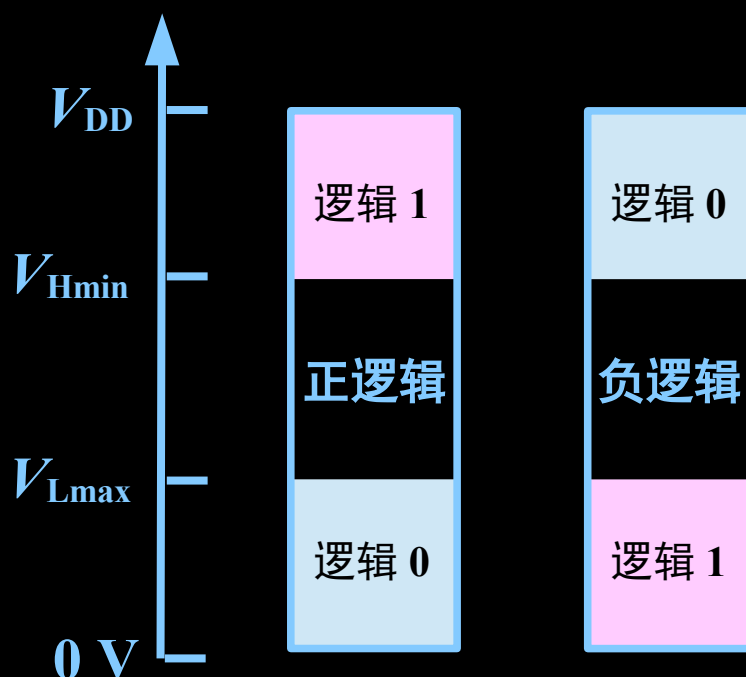
1. 正负逻辑的规定

在数字系统中，可用两种逻辑体制表示电路输入和输出的高、低电平

正逻辑体制：将高电平用逻辑 1 表示，低电平用逻辑 0 表示

负逻辑体制：将高电平用逻辑 0 表示，低电平用逻辑 1 表示

若无特别说明，一般采用正逻辑体制



3.7.1 正负逻辑问题

2. 正负逻辑等效变换

逻辑内涵一致（电路功能一致）的正、负逻辑表达式是等效的

输入 / 输出电平关系

A	B	F
L	L	H
L	H	H
H	L	H
H	H	L

正逻辑

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

$$F = \overline{A \cdot B}$$

负逻辑

A	B	F
1	1	0
1	0	0
0	1	0
0	0	1

$$F = \overline{A + B}$$

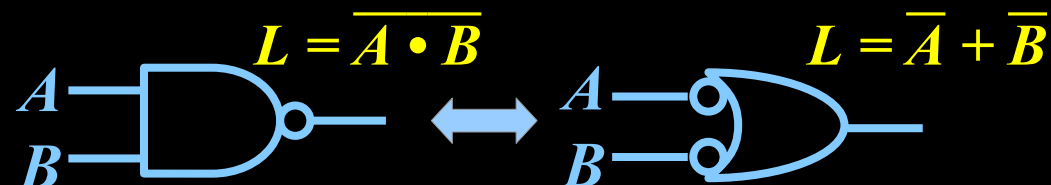
与非 $\longleftrightarrow$ 或非
与 $\longleftrightarrow$ 或

3.7.2 门电路的等效符号及其应用

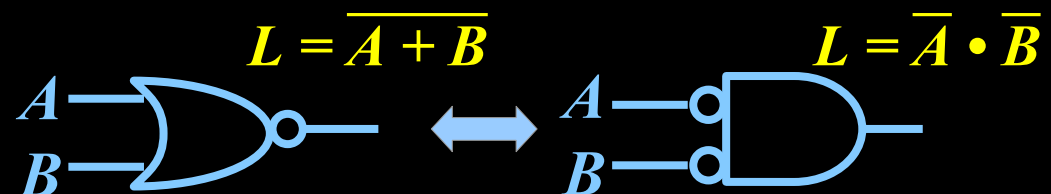
1. 基本逻辑门电路的等效符号

同一逻辑体制下，逻辑内涵一致的逻辑门电路符号，称为等效符号

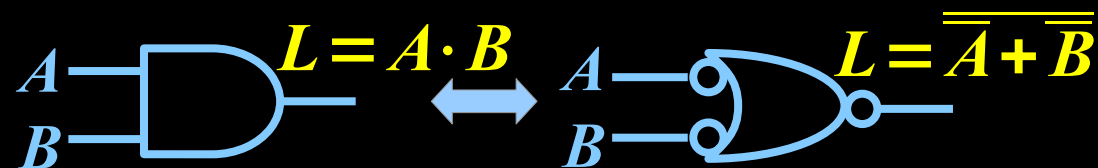
与非门及等效符号如图



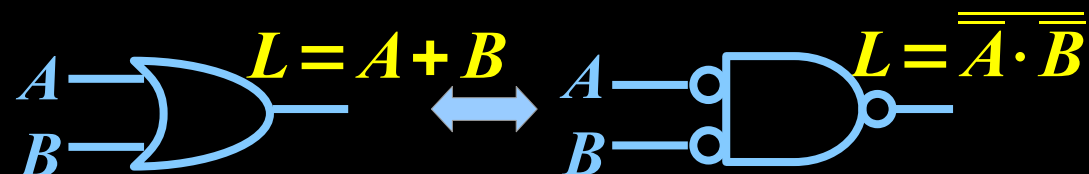
或非门及等效符号如图



与门及等效符号如图



或门及等效符号如图



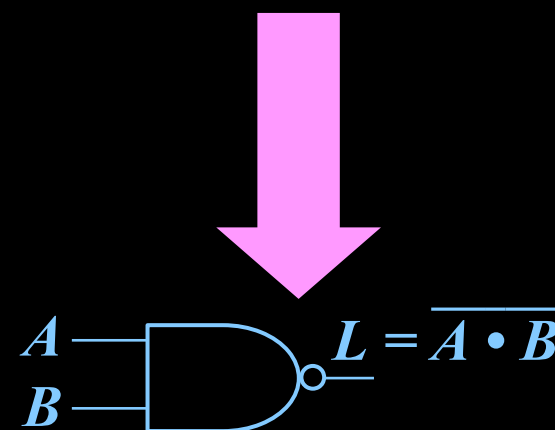
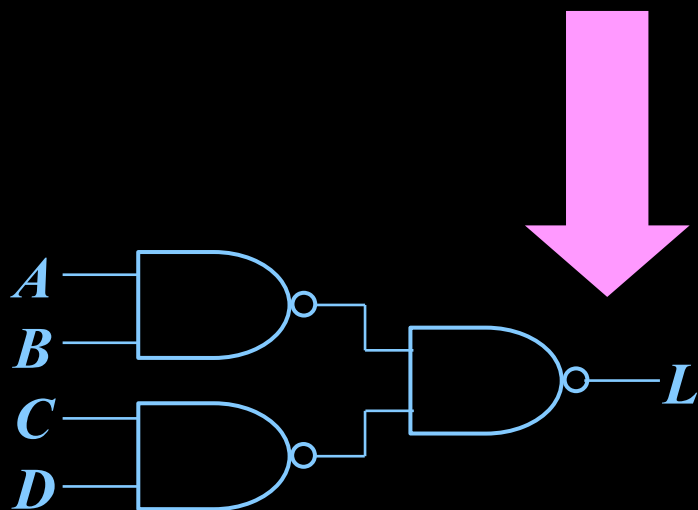
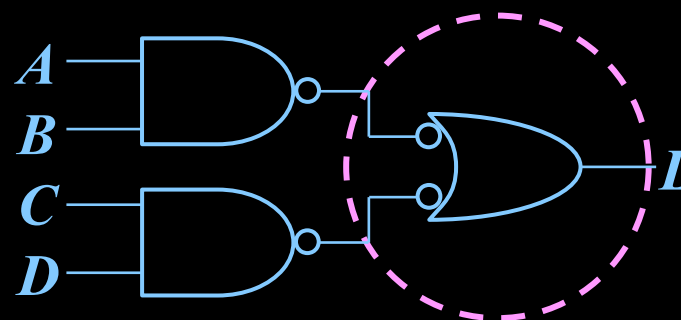
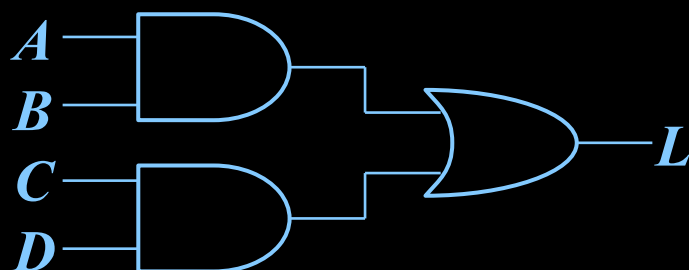
3.7.2 门电路的等效符号及其应用

2. 逻辑门等效符号的应用

利用逻辑门等效符号，可直接变换逻辑电路

$$L = A \cdot B + C \cdot D = \overline{\overline{A \cdot B + C \cdot D}} = \overline{\overline{A \cdot B} \cdot \overline{C \cdot D}}$$

$$L = \overline{A} + \overline{B} = \overline{A \cdot B}$$



3.7.2 门电路的等效符号及其应用

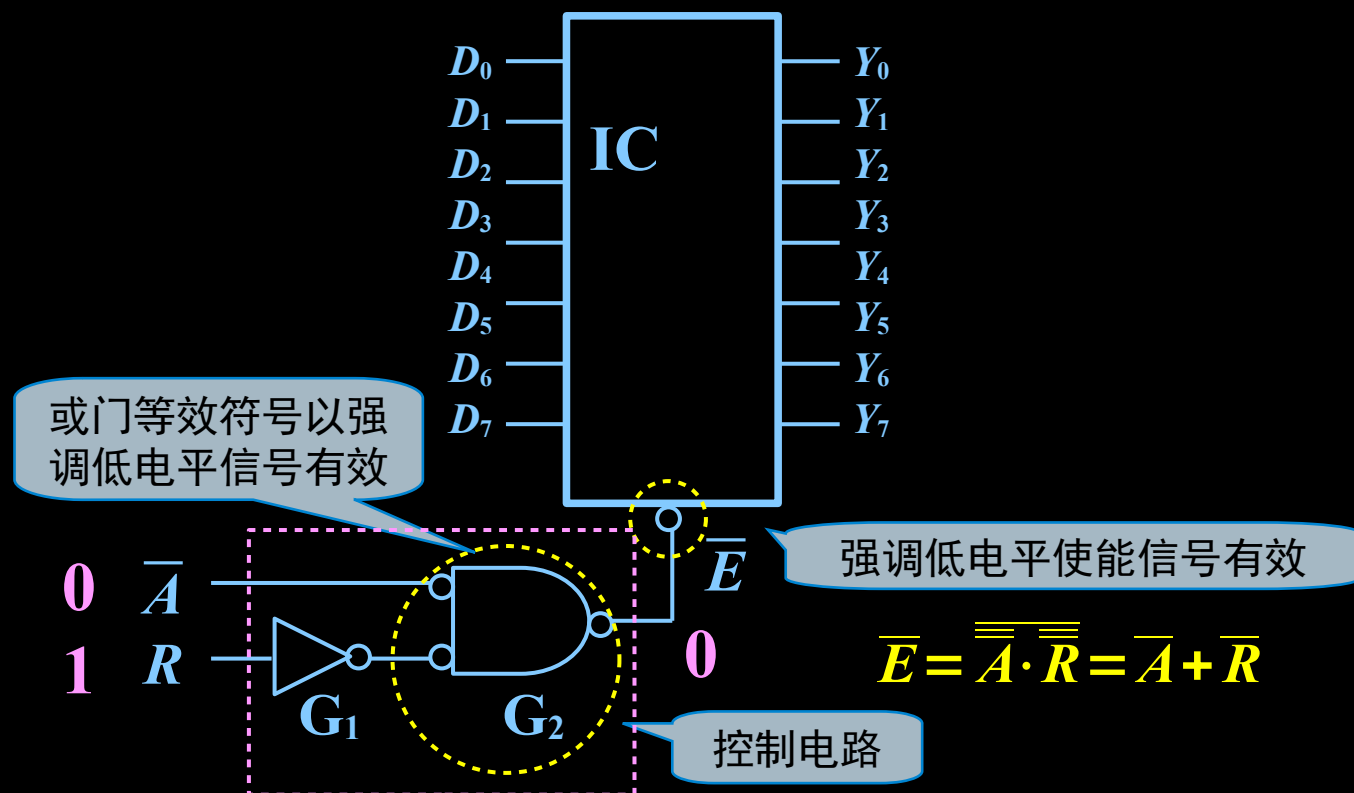
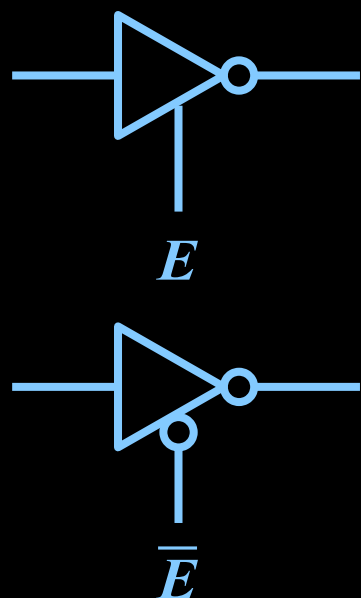
3. 逻辑门等效符号可强调有效电平，便于理解信号高低电平作用

低电平有效：信号低电平时，电路履行预期功能，用反变量形式表示

高电平有效：信号高电平时，电路履行预期功能，用原变量形式表示

如：高电平有效的三态非门， $E = 1$ 时，是非门，否则输出是高阻态

低电平有效的三态非门， $\bar{E} = 0$ 时，是非门，否则输出是高阻态



$R = 1, \bar{A} = 0$ 时， G_2 输出 $\bar{E}$ 有效低电平

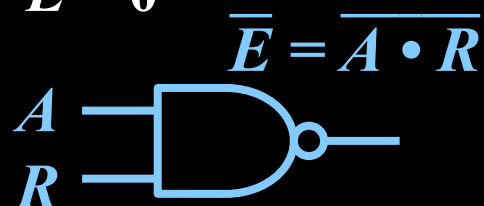
3.7.2 门电路的等效符号及其应用

3. 逻辑门等效符号可强调有效电平，便于理解信号高低电平作用
可根据不同控制信号要求，用等效符号描述有效控制信号

要求

$$R = 1, A = 1$$

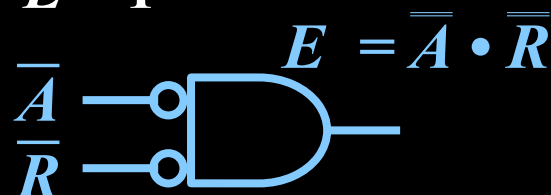
$$\bar{E} = 0$$



要求

$$\bar{R} = 0, \bar{A} = 0$$

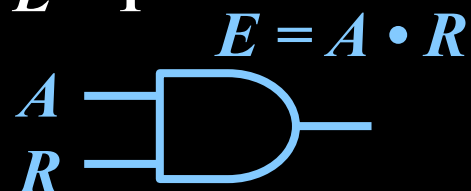
$$E = 1$$



要求

$$R = 1, A = 1$$

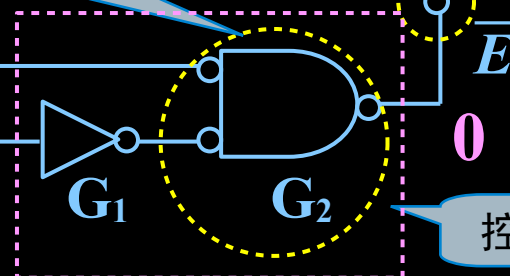
$$E = 1$$



或门等效符号以强调低电平信号有效

0
1

$\bar{A}$
R



控制电路

强调低电平使能信号有效

$$\bar{E} = \overline{\bar{A} \cdot R} = \bar{A} + R$$

$R = 1, \bar{A} = 0$ 时, G_2 输出 $\bar{E}$ 有效低电平

康华光教材 7 版

2.1.1

2.1.2

2.2.7

2.3.4

2.4.1

2.4.3

2.5.6